

***Facultad
de
Ciencias***

**CEREBRO DIGITAL
(DIGITAL BRAIN)**

Trabajo de Fin de Grado
para acceder al

GRADO EN INGENIERÍA INFORMÁTICA

Autora: Estela Simón Fernández

Directora: Marta Elena Zorrilla Pantaleón

Co-Director: Nicolás García Martín

Junio – 2026

RESUMEN

Las aplicaciones de toma de notas permiten almacenar grandes cantidades de información de forma sencilla. Sin embargo, la mayoría de ellas se centran principalmente en el almacenamiento y la recuperación de contenido, ofreciendo un soporte limitado para representar y explotar las relaciones existentes entre las distintas notas.

El presente trabajo de fin de grado aborda el diseño y desarrollo de **Cerebro Digital**, una aplicación orientada a la gestión del conocimiento personal (Personal Knowledge Management, PKM). El sistema emplea archivos en formato *markdown* como mecanismo de almacenamiento y construye una representación estructurada mediante un grafo de conocimiento que modela las relaciones entre notas, enlaces y etiquetas.

Como elemento diferenciador, la aplicación integra modelos de lenguaje grandes (Large Language Models, LLM) mediante un enfoque de generación aumentada por recuperación (Retrieval-Augmented Generation, RAG), permitiendo realizar consultas en lenguaje natural limitando el contexto de generación al contenido introducido por el propio usuario. Este tratamiento posibilita la consulta semántica del conocimiento personal, manteniendo el control sobre la fuente de información utilizada para generar las respuestas.

Cerebro Digital se plantea como un sistema que combina almacenamiento estructurado, modelado explícito de relaciones y consulta asistida mediante lenguaje natural, transformando un repositorio de notas en un entorno activo de exploración y análisis del conocimiento personal.

PALABRAS CLAVE: Gestión del conocimiento personal, grafo de conocimiento, modelos de lenguaje de gran tamaño (LLM), generación aumentada por recuperación

ABSTRACT

The current context, marked by the growing digitalization of information, has made it easier to store data, but has also made it more difficult to organize, link and reuse it in a structured way over time. In response, many note-taking applications prioritize data persistence over the explicit representation of relationships between concepts and their semantic exploitation.

This final year project focuses on the design and development of **Cerebro Digital**, an application geared towards personal knowledge management. The system uses markdown files as a storage mechanism and constructs a structured representation via a knowledge graph that models the relationships between notes, links and tags.

As a key differentiating feature, the application integrates large language models (LLM) using a retrieval augmented generation (RAG) approach, enabling natural language queries whilst limiting the generation context to content entered by the user. This approach facilitates semantic querying of personal knowledge, whilst maintaining control over the source of information used to generate responses.

Cerebro Digital is designed as a system that combines structured storage, explicit relationship modelling and natural language-assisted querying, transforming a repository of notes into an active environment for exploring and analyzing personal knowledge.

KEYWORDS: Personal knowledge management, knowledge graph, large language models (LLM), retrieval augmented generation

ÍNDICE GENERAL

Capítulo 1. Introducción	6
1.1. Motivación.....	6
1.2. Objetivos de desarrollo	7
1.3. Organización de la memoria.....	7
Capítulo 2. Metodología, planificación y herramientas	9
2.1. Metodología de desarrollo	9
2.2. Planificación temporal.....	10
2.3. Herramientas utilizadas	10
2.3.1. Stack tecnológico.....	10
2.3.2. Herramientas de desarrollo	11
Capítulo 3. Análisis y especificación de requisitos	13
3.1. Descripción del sistema	13
3.2. Requisitos funcionales.....	13
3.3. Requisitos no funcionales.....	15
Capítulo 4. Arquitectura y diseño del sistema	16
4.1. Diseño arquitectónico	16
4.2. Organización en capas	17
4.3. Flujo de interacción entre capas	18
4.4. Diseño detallado	20
Capítulo 5. Implementación	22
5.1. Backend	22
5.1.1. Entities.....	22
5.1.2. Repositories	24
5.1.3. Services.....	26
5.1.4. Controllers	27
5.2. Frontend.....	27
5.2.1. Elección tecnológica.....	27
5.2.2. Estructura y organización	28
5.2.3. Comunicación con el backend.....	30
5.2.4. Gestión del estado.....	30
5.2.5. Componentes destacados	31
5.2.6. Integración con el backend	31
Capítulo 6. LLM y enfoque RAG.....	33

6.1. Fundamentos y componentes del sistema LLM	33
6.2. Diseño del pipeline	34
6.3. Generación Aumentada por Recuperación (RAG)	34
6.3.1. Algoritmo de recuperación	35
6.4. Arquitectura de integración	37
Capítulo 7. Verificación y Validación del Software.....	40
7.1. Pruebas unitarias.....	40
7.1.1. Prácticas de implementación en Go.....	40
7.1.2. Pruebas de concurrencia y sincronización.....	41
7.2. Pruebas de integración.....	42
7.2.1. Mocks	42
7.2.2. Diseño de Test Fixtures	42
7.2.3. Casos de sincronización	43
7.2.4. Inyección de generador.....	44
Capítulo 8. Conclusiones y líneas futuras de trabajo	45
8.1. Aportación personal y aprendizaje adquirido	46

ÍNDICE DE FIGURAS

Figura 1 - Diagrama de Gantt	10
Figura 2 - Diagrama de arquitectura del sistema	18
Figura 3 - Diagrama de secuencia	19
Figura 4 - Diagrama de clases UML	20
Figura 5 - Diagrama de componentes	21
Figura 6 - Estructura del proyecto	22
Figura 7 - Vista principal de Cerebro Digital	28
Figura 8 - Vista de detalle de una nota	29
Figura 9 - Subgrafo de conexiones de una nota	29
Figura 10 - Vista del asistente global	30

ACRÓNIMOS

AVL: Análisis de Valores Límite

API REST: Representational State Transfer Application Programming Interface

BSON: Binary JSON

BFS: Breath First Search

CLI: Command Line Interface

CSS: Cascading Style Sheets

HMR: Hot Module Replacement

HTTP: Hypertext Transfer Protocol

IDE: Integrated Development Environment

JSON: JavaScript Object Notation

LLM: Large Language Models

PKM: Personal Knowledge Management

RAG: Retrieval Augmented Generation

UML: Unified Modeling Language

URL: Uniform Resource Locator

WSL: Windows Subsystem for Linux

Capítulo 1. Introducción

El presente trabajo de fin de grado, realizado en colaboración con la empresa **HP SCDS**, se centra en el diseño y desarrollo de una plataforma orientada a la **Gestión del Conocimiento Personal** (Personal Knowledge Management, PKM). El proyecto surge con el objetivo de superar los enfoques convencionales de almacenamiento y gestión de notas, proponiendo un sistema capaz de transformar esta información aislada en una estructura relacional navegable y consultable.

Para ello se hará uso de dos tecnologías clave: el modelado de datos mediante un **grafo de conocimiento** y un mecanismo de **recuperación aumentada mediante generación (RAG)**. Esta combinación permite que el almacenamiento de notas en formato *markdown* no sea el fin del proceso, sino el punto de partida para una interacción asistida entre el usuario y su propia información.

1.1. Motivación

La motivación de este trabajo surge de la limitación observada en las herramientas convencionales de toma de notas y gestión de información, que aunque facilitan el almacenamiento y la recuperación básica de datos, suelen apoyarse en estructuras jerárquicas rígidas como carpetas que obligan al usuario a recordar la ubicación de una idea, en vez de su contexto.

Sin embargo, desde una perspectiva estructural, el conocimiento humano no se organiza de forma lineal ni jerárquica, sino como una red de conceptos interrelacionados que evolucionan con el tiempo. Por ello, el uso de grafos como modelo de representación resulta más adecuado para este propósito, facilitando la identificación de conexiones transversales que a menudo pueden quedar ocultas en listas o carpetas.

Por otro lado, la irrupción de grandes modelos de lenguaje (LLM) ha modificado la forma de interacción de los usuarios con las aplicaciones, favoreciendo las consultas en lenguaje natural. Sin embargo, el uso genérico de estos conlleva riesgos en términos de veracidad y privacidad debido a la ausencia de control explícito sobre el origen de los datos usados para su entrenamiento. Frente a esto, el enfoque RAG actúa como mecanismo de control que reduce el riesgo de generación de respuestas no contrastadas.

1.2. Objetivos de desarrollo

El objetivo principal de este proyecto es diseñar y desarrollar una aplicación de escritorio que automatice la gestión y explotación semántica de las notas personales. Para lograrlo, se establecen los siguientes objetivos más específicos:

- **Gestión de datos estructurados:** definir un mecanismo de diseño y almacenamiento basado en *markdown*, priorizando la portabilidad de los datos y la compatibilidad en múltiples plataformas.
- **Construcción del grafo de conocimiento:** desarrollar un módulo para la extracción automatizada de metadatos y relaciones (enlaces y etiquetas) que posibilite la exploración relacional del conjunto de información.
- **Integración de inteligencia artificial:** configurar un *pipeline* RAG de recuperación y procesamiento de consultas en lenguaje natural, limitando el contexto de generación a las notas proporcionadas por el usuario.
- **Evaluación y análisis:** realizar un estudio sobre el sistema resultante para evaluar su eficacia como herramienta de apoyo y consulta activa del conocimiento personal, identificando sus capacidades, limitaciones y posibles líneas de mejora.

1.3. Organización de la memoria

Después de este capítulo de introducción, la memoria se **¿?** del siguiente modo:

- En el capítulo 2 se describe la metodología de trabajo, la planificación temporal y las herramientas utilizadas durante el desarrollo.
- El capítulo 3 aborda el análisis y especificación de requisitos, así como el alcance del proyecto.
- En el capítulo 4 se presenta y detalla la arquitectura software adoptada.
- El capítulo 5 detalla el proceso de implementación, tanto del *backend* como del *frontend*.
- El capítulo 6 aborda la integración del modelo de lenguaje y el enfoque RAG implementado para la recuperación contextual de información.
- Posteriormente, el capítulo 7 recoge la fase de pruebas realizadas para validar el correcto funcionamiento de la aplicación.

-
- Finalmente, el capítulo 8 expone las conclusiones obtenidas del trabajo y plantea posibles líneas futuras de evolución.

Capítulo 2. Metodología, planificación y herramientas

En este capítulo se expone la metodología de desarrollo seguida, la planificación temporal del proyecto y la delimitación técnica del alcance, así como las tecnologías y herramientas utilizadas para llevar a cabo su implementación y validación.

2.1. Metodología de desarrollo

El desarrollo del proyecto se llevó a cabo siguiendo una metodología ágil inspirada en Scrum, adaptada a las características de un trabajo de fin de grado desarrollado por una única persona. La elección de este enfoque respondió a la naturaleza evolutiva del proyecto, ya que en un principio solo estaban definidos los objetivos generales del sistema, mientras que una gran parte de las decisiones técnicas debían evaluarse y concretarse a medida que avanzaba el desarrollo.

La adopción de una metodología iterativa permitió dividir el sistema en módulos funcionales independientes, como la gestión de notas, la persistencia de datos, la gestión del grafo o la integración con modelos de lenguaje. Estos módulos pudieron implementarse y validarse progresivamente, facilitando la detección temprana de problemas de diseño y la incorporación de mejoras derivadas de las pruebas realizadas. Asimismo, este enfoque permitió ampliar gradualmente el alcance inicial del proyecto, incorporando funcionalidades que no estaban previstas en las primeras fases, como el desarrollo de la interfaz gráfica.

Aunque el proyecto fue desarrollado individualmente, se mantuvieron los principales roles definidos por Scrum. Mis tutores desempeñaron un papel equivalente al Product Owner, proporcionando orientación sobre los objetivos del proyecto, revisando los avances y validando las decisiones adoptadas durante el desarrollo. Mientras que sobre mi recayó de manera simultánea las funciones de desarrollador y analista, siendo responsable de la definición técnica, implementación y validación del sistema.

La planificación se estructuró en iteraciones de aproximadamente dos semanas de duración. Al inicio de cada *sprint* se establecían los objetivos prioritarios y las funcionalidades a desarrollar, mientras que al finalizar se realizaban reuniones de seguimiento para evaluar los avances alcanzados, analizar posibles incidencias y redefinir las prioridades de las siguientes

iteraciones. GitLab se utilizó como repositorio remoto de código y herramienta de control de versiones, permitiendo mantener la trazabilidad de los cambios realizados durante todo el ciclo de desarrollo. En la figura 1 se muestra, mediante un diagrama de Gantt, la planificación temporal de las tareas desarrolladas en cada sprint.

2.2. Planificación temporal

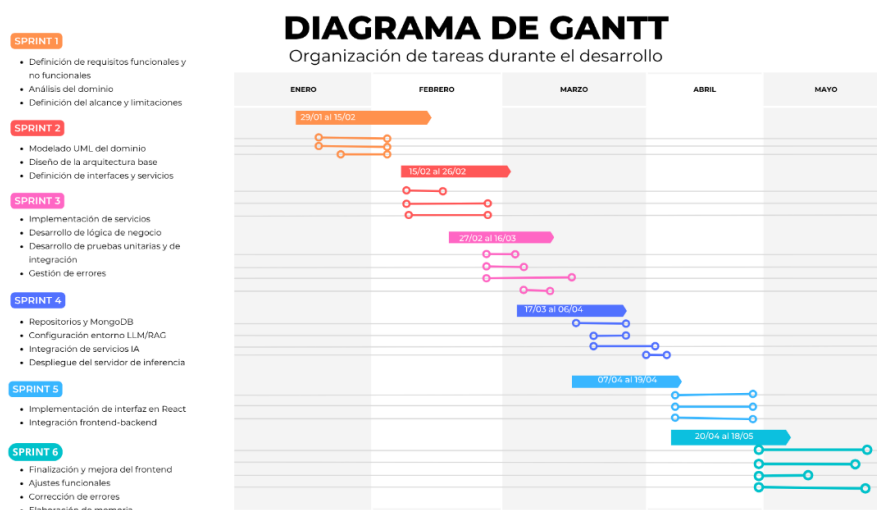


Figura 1 - Diagrama de Gantt

2.3. Herramientas utilizadas

Para el desarrollo de Cerebro Digital se ha seleccionado un conjunto de tecnologías que garantizan la eficiencia en el procesamiento de lenguaje natural y la robustez en la gestión de datos.

2.3.1. Stack tecnológico

Esta sección describe las tecnologías que forman parte de la arquitectura de ejecución del sistema.

○ Go (Golang)

Utilizado como lenguaje principal para el desarrollo del *backend*. Su elección se fundamenta en su simplicidad, rendimiento y facilidad. Esta tecnología fue seleccionada inicialmente en la propuesta del TFG. Además, se empleó el paquete estándar *testing* para la implementación de pruebas unitarias y de integración.

○ MongoDB

Base de datos documental NoSQL utilizada para la persistencia. Se seleccionó por su modelo de datos flexible basado en JSON que permite fácilmente y de manera más natural modelar las relaciones entre conceptos.

- **Markdown**

Estándar de formato de texto plano adoptado para garantizar la portabilidad y legibilidad de la información a largo plazo.

- **Llama.cpp**

Motor de inferencia para modelos de lenguaje que permite ejecutar modelos LLM de forma local en CPU o GPU de forma eficiente. En este proyecto se utiliza como servidor HTTP compatible con la API de OpenAI, recibiendo peticiones desde el *backend* y generando respuestas en lenguaje natural.

- **Qwen3-4B**

LLM empleado como motor generativo del sistema. Se utiliza en formato GGUF cuantizado a 4 bits (Q4_K_M), lo que reduce significativamente el consumo de memoria e inferencia manteniendo un equilibrio entre rendimiento y calidad de respuesta. Se emplea la variante *thinking*, orientada a tareas de razonamiento y análisis sobre el contexto estructurado recuperado.

- **React y Vite**

Tecnología utilizada para el desarrollo del *frontend* mediante una arquitectura basada en componentes reutilizables y gestión reactiva del estado, facilitando la construcción de la interfaz y su integración con la API REST del *backend*.

2.3.2. Herramientas de desarrollo

Herramientas utilizadas para la construcción, depuración y gestión del ciclo de vida del software:

- **Nix & WSL**

Se ha utilizado Nix para definir un entorno de desarrollo replicable y aislado, lo que posibilita fijar versiones concretas de dependencias como Go o MongoDB y evitando conflictos entre configuraciones. Para su ejecución en Windows, este entorno se apoya en *Windows Subsystem for Linux* (WSL).

- **Visual Studio Code**

Entorno de desarrollo integrado (IDE) utilizado tanto para la implementación del *backend* como del *frontend*. Su elección se debe a su ligereza, facilidad de uso y amplia disponibilidad de extensiones. Entre las utilizadas destacan Go para el desarrollo en Golang y Live Server para el desarrollo con React.

- **Git & GitLab**

Como se ha mencionado en la metodología, Git actúa como sistema de control de versiones local, mientras que [GitLab](#) sirve como repositorio remoto y plataforma de gestión del proyecto.

- **StarUML**

Herramienta de diseño utilizada para la creación de los distintos diagramas UML que forman parte de la documentación del proyecto.

- **Docker**

Utilizado para establecer un contenedor y poder generar el despliegue del sistema, permitiendo empaquetar la aplicación y sus dependencias de ejecución en un entorno portable y reproducible. A diferencia de Nix, orientado al entorno de desarrollo, Docker se emplea para facilitar la distribución, puesta en marcha y despliegue del sistema para terceros.

Capítulo 3. Análisis y especificación de requisitos

En este apartado, se realiza una descripción general de la aplicación y se detallan los requisitos y funcionalidades que esta debe proporcionar para cumplir con los objetivos planteados.

3.1. Descripción del sistema

Cerebro Digital es un sistema orientado a la gestión del conocimiento personal que permite al usuario almacenar, organizar y explotar información de forma estructurada y contextual. La aplicación se basa en la persistencia de archivos *markdown*, los cuales se representan internamente mediante un grafo de conocimiento en el que las notas actúan como nodos mientras que sus relaciones (enlaces) actúan como aristas. Esta estructura facilita una exploración más flexible y contextual de la información.

Sobre esta base, el sistema incorpora un servicio de consulta en lenguaje natural basado en un enfoque RAG. En este proceso, el *backend* recopila contexto relevante mediante la exploración del grafo de conocimiento. Posteriormente, esta información se utiliza para construir consultas enriquecidas que son procesadas por un modelo de lenguaje ejecutado localmente a través de llama.cpp. Este enfoque permite generar respuestas fundamentadas exclusivamente en el contenido proporcionado por el usuario, proporcionando eficiencia, escalabilidad y control sobre la privacidad de los datos.

De este modo, la aplicación no se limita a actuar como un repositorio de información, sino que se configura como un entorno activo que permite analizar, relacionar y recuperar información de forma asistida por medio del RAG, manteniendo la trazabilidad y el control sobre el origen de la información.

3.2. Requisitos funcionales

Los requisitos se han definido de forma concreta, medible y verificable, sirviendo como base para el posterior diseño, desarrollo y validación del sistema.

ID	NOMBRE	DESCRIPCIÓN
Gestión de Notas		
RF01	Gestión básica de notas	El sistema debe permitir al usuario crear, editar y eliminar notas.
RF02	Persistencia de notas	El sistema debe almacenar y recuperar el contenido de las notas.
RF03	Metadatos de contexto	El sistema debe permitir asociar metadatos a las notas para facilitar su organización y posterior filtrado.
Organización y clasificación		
RF04	Gestión de etiquetas	El sistema debe permitir la creación, asignación y eliminación de etiquetas como mecanismo flexible de clasificación de notas.
Relación entre conocimientos		
RF05	Enlaces manuales entre notas	El sistema debe permitir relaciones explícitas entre notas mediante enlaces internos definidos por el usuario.
RF06	Mantenimiento de enlaces	El sistema debe mantener actualizados los enlaces, aunque una nota sea renombrada.
Estructura de almacenamiento		
RF07	Gestión del grafo de conocimiento	El sistema debe mantener internamente un grafo, donde las notas actúan como nodos y los enlaces como aristas, garantizando su creación, actualización y eliminación de forma correcta.
RF08	Navegación y consulta del grafo	El sistema debe permitir la exploración de relaciones entre notas mediante la obtención de conexiones entrantes y salientes, así como la generación de subgrafos.
Búsqueda y consulta de información		
RF09	Búsqueda textual sobre el conocimiento	El sistema debe permitir realizar búsquedas textuales sobre el contenido de las notas para facilitar la recuperación de información.

Asistencia mediante modelos de lenguaje		
RF10	Consultas en lenguaje natural	El sistema debe permitir al usuario realizar consultas en lenguaje natural sobre el contenido almacenado en sus notas.
RF11	Respuestas basadas en conocimiento personal	Las respuestas generadas mediante modelos de lenguaje deben basarse exclusivamente en la información contenida en las notas del usuario.

3.3. Requisitos no funcionales

Los requisitos no funcionales describen restricciones y atributos de calidad que debe cumplir Cerebro Digital, complementando los requisitos funcionales previamente definidos.

ID	TIPO	DESCRIPCIÓN
RNF01	Usabilidad	El sistema debe ofrecer mecanismos de interacción simples y comprensibles, priorizando la claridad funcional sobre la complejidad visual.
RNF02	Fiabilidad	El sistema debe garantizar la persistencia de las notas ante cierres inesperados.
RNF03	Recuperación ante fallos	El sistema debe ser capaz de recuperar el estado consistente tras un fallo.
RNF04	Seguridad	El sistema no debe compartir el contenido sin consentimiento explícito del usuario.
RNF05	Seguridad	El sistema debe proteger los datos del usuario frente a accesos no autorizados.
RNF06	Portabilidad	El sistema debe poder ejecutarse en entornos Windows mediante WSL.
RNF07	Rendimiento	El sistema debe generar resultados de búsqueda en un tiempo inferior a 10 segundos en condiciones normales.
RNF08	Rendimiento	El sistema debe poder gestionar un conjunto de al menos 2000 notas sin aumento superior al 30% en el tiempo de respuesta.

Capítulo 4. Arquitectura y diseño del sistema

En este capítulo se expone la estructura técnica y las decisiones de diseño que fundamentan el desarrollo de **Cerebro Digital**. Se describe el modelo arquitectónico adoptado, la organización entre las distintas capas y la interacción entre los distintos módulos del sistema.

4.1. Diseño arquitectónico

La arquitectura del sistema se basa en los principios de **Clean Architecture**, propuestos por Robert C. Martin [1]. Este modelo arquitectónico promueve una clara separación de responsabilidades entre los distintos componentes del software mediante una organización en capas donde las dependencias siempre apuntan hacia el núcleo del sistema.

El objetivo principal de este enfoque es **desacoplar la lógica de negocio de los detalles tecnológicos**, como el acceso a base de datos, *frameworks* o servicios externos. De este modo, el núcleo de la aplicación permanece independiente de las tecnologías utilizadas en su implementación, lo que favorece la evolución, mantenibilidad y testeabilidad del sistema a largo plazo. El sistema aplica el principio de inversión de dependencias, de forma que los detalles de bajo nivel (como MongoDB o el cliente LLM) dependen de abstracciones definidas en capas internas.

Clean Architecture se basa en 3 principios fundamentales:

- **Independencia de *frameworks*:** el sistema no depende de librerías externas para su estructura central.
- **Testeabilidad:** la lógica de negocio puede probarse de forma aislada.
- **Separación de responsabilidades:** cada capa tiene un propósito específico dentro del sistema.

En el caso de Cerebro Digital, este enfoque resulta especialmente adecuado debido a la coexistencia de múltiples componentes, como la base de datos MongoDB y la integración con modelos de lenguaje. Mediante esta arquitectura se evita que estos elementos afecten directamente al núcleo del dominio.

4.2. Organización en capas

Como se puede observar en la figura 2, la arquitectura se organiza en capas concéntricas, donde las dependencias únicamente apuntan hacia las capas internas. A continuación, se detallan las capas que conforman el sistema:

- **Capa de presentación (CLI/API REST):** es la capa más externa y desempeña el rol de punto de acceso para el usuario. Es “agnóstica” a la información que transporta y su única responsabilidad es gestionar la comunicación con el cliente, ya sea mediante una línea de comandos (CLI) o con una API REST mediante peticiones HTTP.
- **Capa de controladores:** actúa como mediador o adaptador en el sistema de comunicación. Su función principal es desacoplar la capa de aplicación de los detalles del protocolo de comunicación (HTTP). Se encarga de describir los *endpoints* y cómo se convierten las peticiones en llamadas a servicios de aplicación.
- **Capa de aplicación:** contiene los casos de uso y la lógica de aplicación del sistema. Su responsabilidad es coordinar la interacción entre las entidades del dominio y los servicios de infraestructura. En esta capa se localizan los siguientes servicios fundamentales:
 - **NoteService:** encargado de gestionar el ciclo de vida de las notas.
 - **LinkService:** encargado de la creación y mantenimiento de los enlaces entre notas.
 - **GraphService:** responsable de gestionar el flujo de datos hacia y desde las entidades de dominio sin conocer los detalles de su almacenamiento.
 - **RagService:** implementa el *pipeline* RAG, actuando como orquestador del caso de uso de consulta en lenguaje natural. Coordina la recuperación de contexto desde repositorios y la generación de inferencia a través del cliente LLM.

Además, se emplea un **ServiceContainer** que facilita la inyección de dependencias, permitiendo centralizar la creación y configuración de los servicios.

Estos servicios utilizan las interfaces definidas en el dominio para acceder a los datos, lo que permite mantener el desacoplamiento entre la lógica de negocio y la infraestructura.

- **Capa de dominio:** constituye el núcleo del sistema y la capa más interna. Contiene las entidades básicas del negocio, así como las definiciones de las interfaces que representan el contrato para el acceso a datos. El dominio no tiene conocimiento sobre la existencia de la base de datos ni del LLM.
- **Capa de infraestructura:** contiene las implementaciones concretas de los repositorios definidos en el dominio para el almacenamiento de notas y enlaces en MongoDB. Además, esta capa incluye el cliente encargado de la comunicación con el servicio externo (cliente LLM). Depende de la capa de dominio.

Gracias a la separación proporcionada por Clean Architecture, estos componentes pueden modificarse sin afectar al resto del sistema.

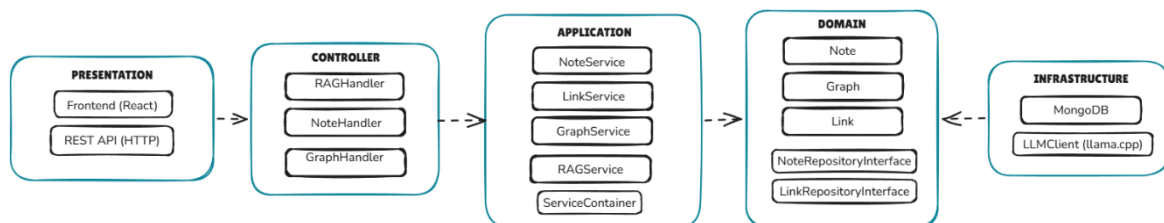


Figura 2 - Diagrama de arquitectura del sistema

4.3. Flujo de interacción entre capas

El flujo de interacción entre capas sigue el principio de dependencia hacia el interior: las capas externas dependen de las internas, pero nunca al contrario.

A continuación, y como se ilustra en la figura 3, se describe el flujo de interacción seguido para realizar una consulta al sistema RAG:

1. El usuario realiza una petición HTTP a través del *frontend*, enviando una pregunta al endpoint */api/ask*.
2. La petición es procesada por el controlador, que recibe la petición, valida los datos de entrada (decodificando el cuerpo JSON) y delega la ejecución en el servicio correspondiente.
3. El servicio ejecuta la lógica de negocio utilizando las entidades del dominio sin conocer los detalles de implementación:
 - Decide si la consulta es contextual (a partir de una nota) o global.

- Si es contextual, explora el grafo para encontrar notas relacionadas mediante enlaces y etiquetas.
 - Si es global, delega en el repositorio la búsqueda de notas relevantes.
4. Si es necesario acceder a datos persistentes, el servicio utiliza las interfaces de repositorio definidas en la capa de dominio.
 5. La implementación concreta de dichas interfaces se encuentra en la capa de infraestructura, que ejecuta las consultas contra MongoDB utilizando índices de texto y etiquetas. Para la generación de respuestas, el servicio utiliza la interfaz LLMClient (definida en aplicación) sin conocer su implementación. La capa de infraestructura proporciona el cliente que se comunica con el servidor *llama.cpp* a través de HTTP.

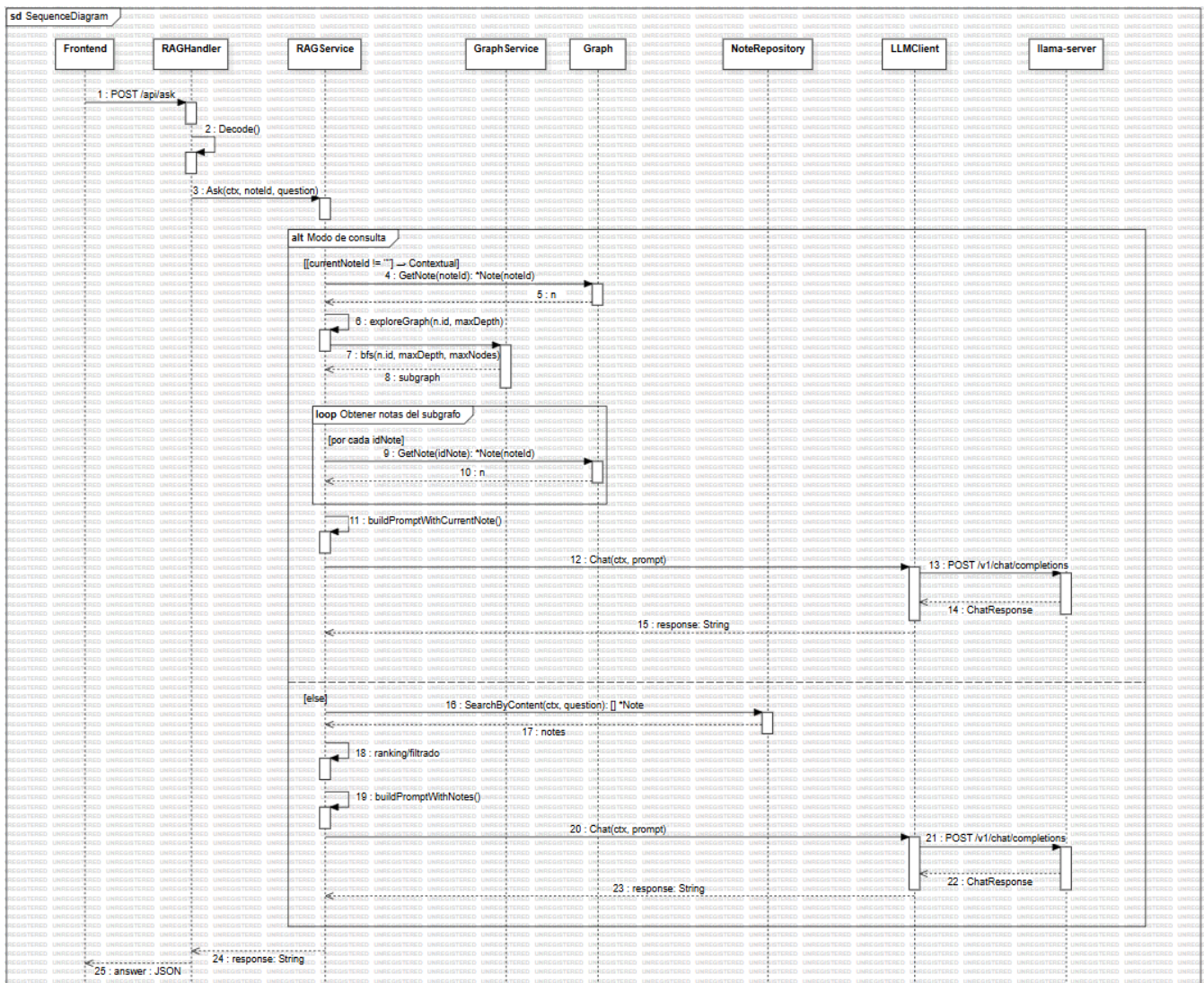


Figura 3 - Diagrama de secuencia

4.4. Diseño detallado

Una vez definida la arquitectura general del sistema, se procede a describir con mayor nivel de detalle la estructura interna de los componentes que forman parte de la aplicación. Para ello, se han elaborado distintos diagramas UML que permiten representar de forma visual las relaciones entre los elementos del sistema. En primer lugar, la figura 4 presenta un diagrama de clases, en el que se representan las entidades del dominio y sus relaciones. Este diagrama permite comprender la estructura del modelo de datos, así como las relaciones existentes entre las distintas clases que conforman la lógica del sistema.

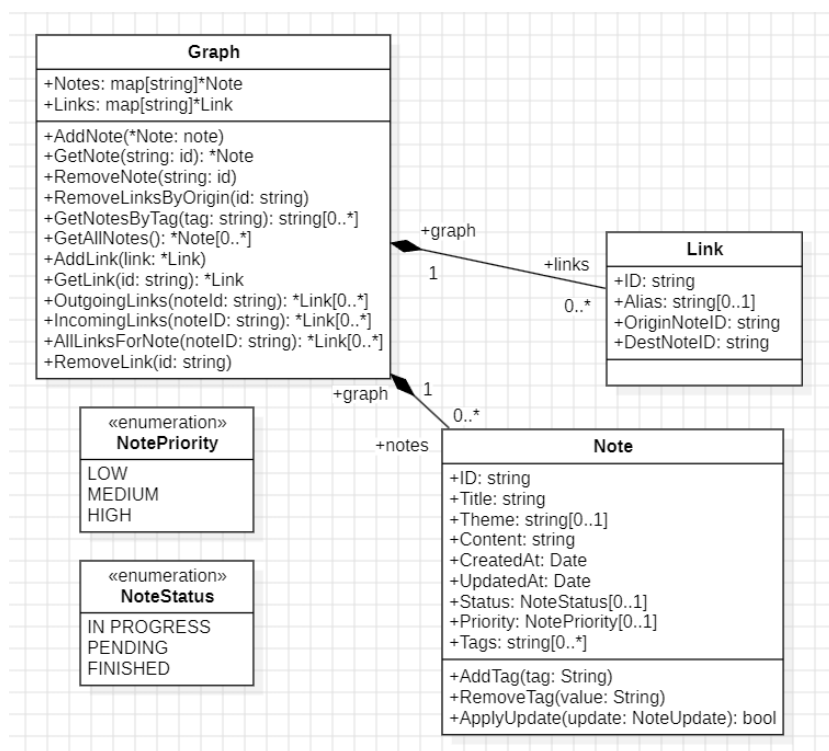


Figura 4 - Diagrama de clases UML

Asimismo, como puede observar en la figura 5, se incluye un **diagrama de componentes**, que muestra la organización modular del sistema y las dependencias entre los principales módulos de la aplicación. Esta representación permite visualizar cómo interactúan los distintos servicios de aplicación y los repositorios de infraestructura dentro de la arquitectura definida.

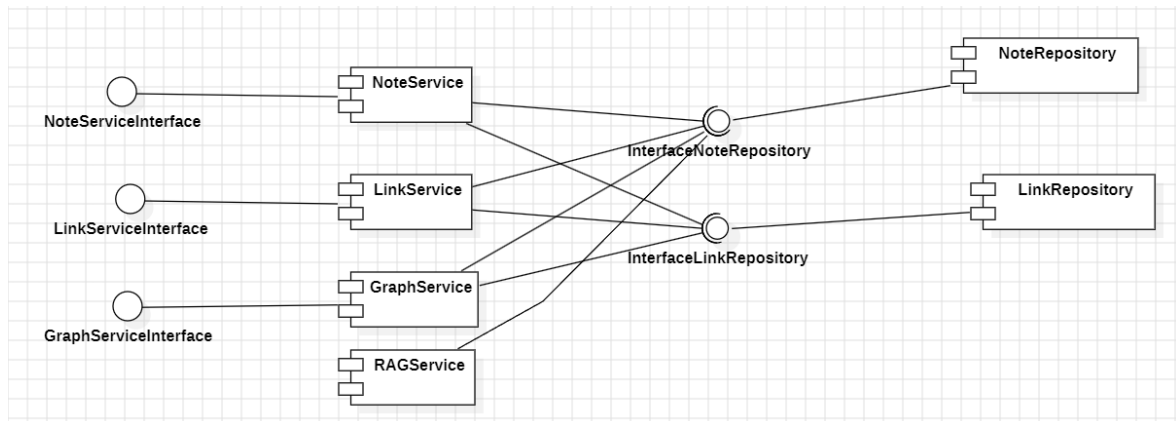


Figura 5 - Diagrama de componentes

Ambos diagramas proporcionan una visión complementaria del diseño del sistema: mientras que el diagrama de componentes describe la organización a nivel arquitectónico, el diagrama de clases permite analizar con mayor detalle la estructura interna de los elementos del dominio y su interacción dentro de la aplicación.

Capítulo 5. Implementación

5.1. Backend

La implementación del sistema se ha realizado utilizando el lenguaje Go, siguiendo la arquitectura descrita en el capítulo anterior. La estructura del proyecto refleja directamente la separación en capas propuesta en el capítulo de diseño, organizando los componentes en módulos independientes para dominio, persistencia, servicios de aplicación y exposición de la API REST.

La figura 6 muestra la organización general del código fuente, donde cada paquete agrupa responsabilidades concretas y mantiene el desacoplamiento entre lógica de negocio e infraestructura.

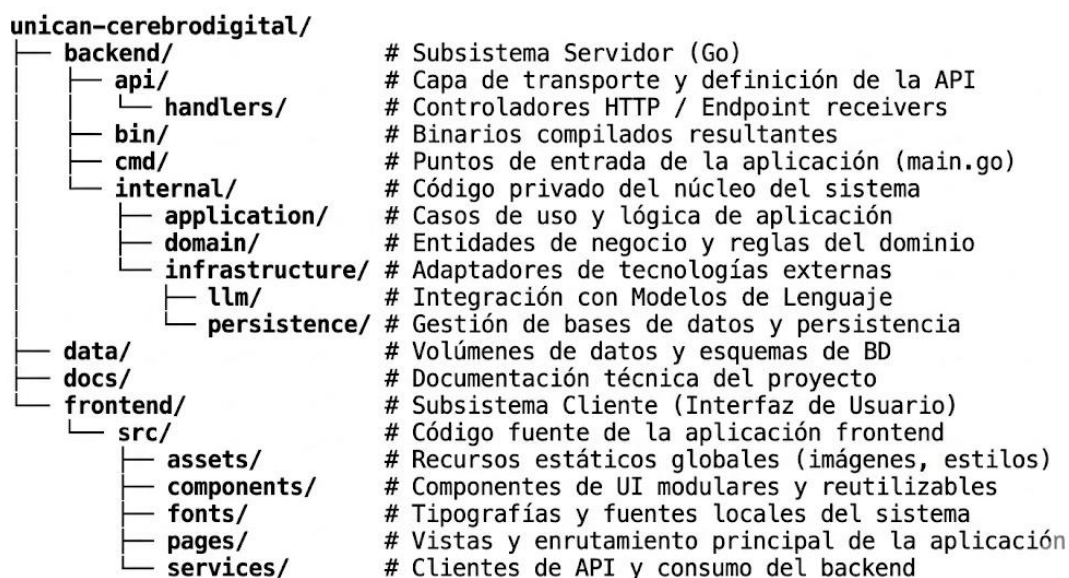


Figura 6 - Estructura del proyecto

En las siguientes secciones se describen los principales elementos implementados y algunas de las decisiones técnicas adoptadas durante el desarrollo.

5.1.1. Entities

Las **entidades de dominio** representan los elementos principales del modelo de conocimiento del sistema. En el caso de Cerebro Digital, las principales entidades implementadas son Note, Link y Graph.

Estas entidades encapsulan tanto los datos como parte de la lógica asociada al dominio, evitando que las reglas del sistema queden dispersas en otras capas de la aplicación.

La definición de las estructuras incluye *struct tags* para facilitar su serialización y almacenamiento en MongoDB. Este mecanismo facilita la persistencia de los datos sin introducir dependencias directas en la lógica de dominio.

- **Note:** representa la unidad básica de conocimiento dentro del sistema. Cada nota contiene información textual junto con metadatos adicionales que permiten su organización y clasificación.
 - Atributos principales:
 - **Id:** identificador único de la nota.
 - **Title y content:** información textual de la nota.
 - **Theme:** temática opcional asociada a la nota.
 - **Status y priority:** metadatos que representan el estado y la prioridad de la nota, por defecto se incluye ‘pendiente’ como estado.
 - **Tags:** etiquetas utilizadas para la clasificación flexible del contenido.
 - **CreatedAt y updatedAt:** marcas temporales que permiten registrar la creación y modificación de la nota.

Para permitir modificaciones parciales de forma eficiente, se define la estructura **NoteUpdate**. Esta estructura emplea punteros en sus atributos para diferenciar entre campos no modificados y cambios explícitos realizados por el usuario, incluso cuando el nuevo valor es vacío.

De este modo, el método `ApplyUpdate()` puede aplicar de manera precisa los cambios solicitados, manteniendo el resto de la información intacta. Además, este método incorpora validaciones adicionales para asegurar que los valores introducidos son válidos dentro del dominio.

- **Link:** representa una relación entre dos notas dentro del sistema. Conceptualmente, estos enlaces permiten modelar conexiones semánticas entre distintos elementos de conocimiento.
 - Atributos principales:
 - **Id :** identificador único del enlace.

- **OriginNoteID:** identificador de la nota origen.
- **DestNoteID:** identificador de la nota destino.
- **Alias:** campo opcional que titula la relación entre ambas notas.
- **Graph:** representa el grafo de conocimiento construido a partir de las notas y los enlaces existentes. A diferencia de **Note** y **Link**, esta estructura no se almacena directamente en la base de datos, sino que se reconstruye dinámicamente en memoria a partir de la información persistida.

Internamente, el grafo mantiene dos estructuras de datos principales: un mapa de notas y otro de enlaces. Estas estructuras permiten realizar consultas eficientes sobre las relaciones existentes entre las distintas notas.

Dado que el sistema se ejecuta como un servidor web capaz de procesar múltiples peticiones simultáneamente, es necesario garantizar el acceso seguro al grafo en memoria.

Para ello, la estructura **Graph** incorpora un **mecanismo de sincronización** basado en `sync.RWMutex`, que permite coordinar operaciones de lectura y escritura evitando que varias *goroutines* modifiquen simultáneamente las estructuras internas del grafo, lo que podría provocar inconsistencias en los datos.

Los **identificadores** de las entidades se generan en la capa de servicios utilizando el tipo `ObjectID` de MongoDB y posteriormente se convierten a representación hexadecimal para su almacenamiento. Este enfoque desacopla la generación de identificadores de la persistencia y facilita las pruebas del sistema.

5.1.2. Repositories

Los repositorios constituyen el mecanismo de acceso a los datos persistentes del sistema. Sus interfaces se definen en la capa de dominio, mientras que sus implementaciones concretas se sitúan en la capa de infraestructura, permitiendo desacoplar la lógica de aplicación de la tecnología de persistencia utilizada.

De esta forma, la aplicación puede operar sobre abstracciones sin depender directamente de MongoDB u otro sistema de persistencia.

Las interfaces de repositorio definen las operaciones que la lógica de aplicación puede realizar sobre las entidades persistentes. En este proyecto se han definido dos repositorios principales ubicados en el paquete `/domain/repositories`:

- **NoteRepository**, encargado de la gestión de las notas.
- **LinkRepository**, encargado de gestionar las relaciones entre notas.

El uso de interfaces permite desacoplar la lógica de aplicación de la tecnología de persistencia, facilitando tanto las pruebas unitarias como la evolución futura del sistema.

La implementación de los repositorios se encuentra en el paquete `/infrastructure/mongodb`, donde se encapsulan todos los detalles específicos del *driver* oficial de MongoDB para Go.

Todas las operaciones utilizan el `context.Context`, el mecanismo estándar en Go para controlar el ciclo de vida de las operaciones, gestionar cancelaciones en operaciones I/O, así como establecer límites de tiempo para operaciones costosas como consultas a la base de datos o llamadas a APIs externas.

La gestión de errores sigue dos principios principalmente:

- **Traducción de errores de infraestructura:** los errores específicos del *driver* de MongoDB se convierten a errores definidos en el dominio para evitar dependencias en capas superiores.
- **Propagación de errores:** los repositorios devuelven los errores sin añadir contexto adicional, dejando esta responsabilidad a la capa de aplicación. Esto permite mantener una clara separación entre:
 - Errores técnicos (infraestructura).
 - Errores de negocio (dominio).

Para implementar las búsquedas textuales sobre las notas se ha utilizado el operador `$text` de MongoDB.

A diferencia de las consultas basadas en expresiones regulares `$regex`, que requieren recorrer todos los documentos de la colección, las búsquedas de texto utilizan índices de texto, lo que mejora significativamente el rendimiento.

El proceso consiste en:

1. Crear un índice de texto sobre los campos relevantes (por ejemplo, `title` y `content`).
2. Ejecutar consultas utilizando el operador `$text`.

Las entidades utilizan *struct tags* BSON para definir su representación en MongoDB y controlar la serialización de los documentos almacenados.

La opción *omitempty* permite omitir el campo cuando está vacío, lo que permitiría que MongoDB genere automáticamente un `ObjectID`. Sin embargo, en este proyecto los identificadores se generan manualmente en la capa de aplicación.

5.1.3. Services

La capa de servicios constituye el núcleo de la **lógica de aplicación** del sistema. Su responsabilidad es coordinar las operaciones del dominio, interactuar con los repositorios y garantizar la consistencia entre el grafo en memoria y la persistencia en base de datos.

Durante la implementación, uno de los aspectos más relevantes ha sido la gestión sincronizada del grafo de conocimiento, así como la integración del *pipeline* RAG con el modelo de lenguaje ejecutado localmente.

Los principales servicios implementados son:

- **GraphService**: reconstruye el grafo en memoria al iniciar la aplicación y proporciona operaciones de navegación y consulta sobre las relaciones entre notas.
- **NoteService**: responsable de las operaciones de creación, actualización y eliminación de notas.
- **LinkService**: encargado de gestionar las relaciones entre notas.
- **RAGService**: integra el grafo de conocimiento con el modelo de lenguaje para recuperar contexto relevante y generar respuestas fundamentadas.

Los servicios se instancian mediante un **contenedor de servicios**, encargado de centralizar la creación de repositorios, clientes externos y componentes compartidos por varios servicios como el grafo en memoria. Este enfoque facilita el desacoplamiento entre componentes y simplifica las pruebas unitarias mediante el uso de implementaciones simuladas (*mocks*).

En particular, **GraphService** reconstruye el grafo cargando las notas y los enlaces almacenados en la base de datos e insertándolos en la estructura de memoria utilizada por la aplicación. Esto permite realizar consultas sobre relaciones entre notas directamente en memo-

ria, sin necesidad de acceder continuamente a MongoDB, mejorando el rendimiento de operaciones como la navegación entre notas conectadas o la obtención de enlaces entrantes o salientes.

Para mantener la consistencia entre la persistencia y el grafo en memoria, las operaciones de creación, actualización y eliminación siguen un patrón de doble escritura: primero se realiza la operación sobre MongoDB y, únicamente si esta finaliza correctamente, se actualiza la estructura en memoria. En caso de error durante la segunda fase, se aplican mecanismos de *rollback* para evitar estados inconsistentes.

Asimismo, para la generación de identificadores únicos de las entidades se utiliza un **generador de IDs** inyectado en los servicios basado en el tipo `ObjectID` de MongoDB, lo que desacopla la lógica de aplicación de la estrategia concreta de generación de IDs y facilita las pruebas.

5.1.4. Controllers

Los **controladores** constituyen el punto de entrada de la aplicación para las peticiones HTTP externas. Su responsabilidad consiste en recibir las solicitudes del cliente, validar los datos de entrada, delegar la lógica en los servicios de aplicación y devolver una respuesta estructurada.

Para representar los datos intercambiados mediante la API se utilizan estructuras específicas de *request* y *response*, actuando como DTOs (Data Transfer Objects) independientes de las entidades de dominio. Este enfoque evita acoplar la representación externa de la API con la lógica interna del sistema y facilita el control del proceso de serialización y deserialización mediante *struct tags* JSON.

5.2. Frontend

A continuación, se describe la implementación de la interfaz de usuario del sistema, desarrollada mediante React y Vite. La aplicación sigue una arquitectura basada en componentes reutilizables combinada con una comunicación asíncrona con el *backend* a través de una API REST, manteniendo separada la lógica de presentación de la lógica de negocio.

5.2.1. Elección tecnológica

Para el desarrollo del *frontend* se ha utilizado React junto con Vite como herramienta de construcción. React permite estructurar la interfaz mediante componentes independientes y reutilizables, facilitando el mantenimiento y la evolución del sistema. Además, el uso de *hooks* como `useState` y `useEffect` simplifica la gestión del estado y del ciclo de vida de los componentes sin necesidad de soluciones de estado global más complejas.

Por su parte, Vite proporciona un entorno de desarrollo ligero con soporte para *Hot Module Replacement* (HMR), permitiendo visualizar cambios en tiempo real sin recargar completamente la aplicación y mejorando la productividad durante el desarrollo.

A nivel visual, la interfaz se ha implementado utilizando CSS con un tema oscuro consistente, orientado a mejorar la legibilidad y reducir la fatiga visual. Se ha optado por un diseño minimalista centrado en el contenido y en la interacción del usuario con las notas.

5.2.2. Estructura y organización

La aplicación se organiza en torno a tres vistas o pantallas principales, gestionadas mediante rutas con `react-router-dom`:

- **Lista de notas:** página principal donde se muestran todas las notas del usuario en formato de tarjetas. Desde esta vista es posible acceder al detalle de cada nota o crear nuevas notas mediante un formulario modal.

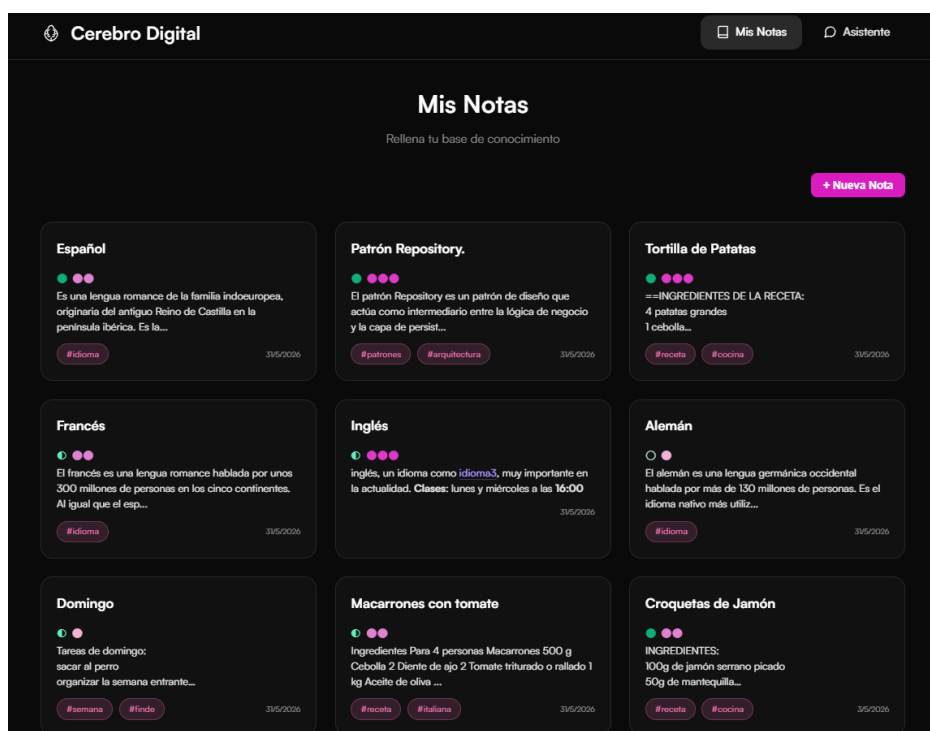


Figura 7 - Vista principal de Cerebro Digital

- **Detalle de nota:** permite visualizar, editar y eliminar una nota. Además, incluye la representación visual del subgrafo de conexiones asociado a esa nota y la posibilidad de realizar consultas contextuales a partir del contenido de la misma.

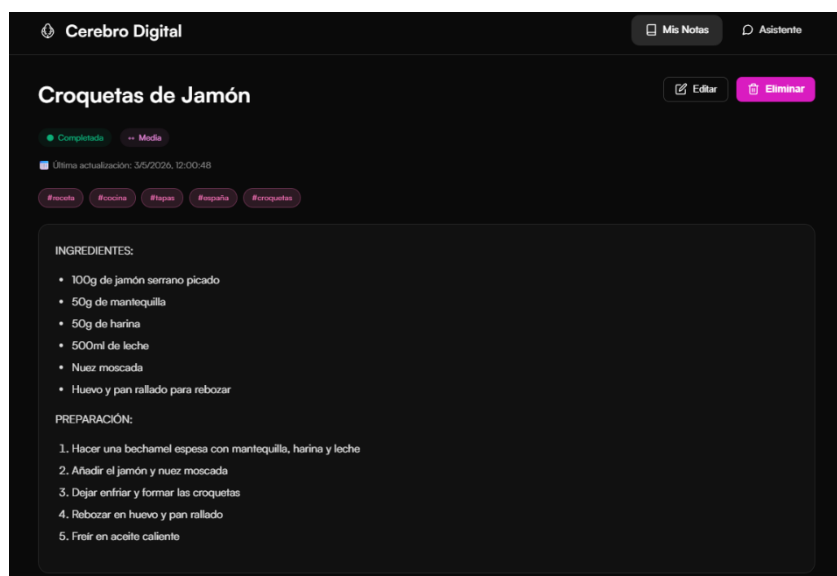


Figura 8 - Vista de detalle de una nota

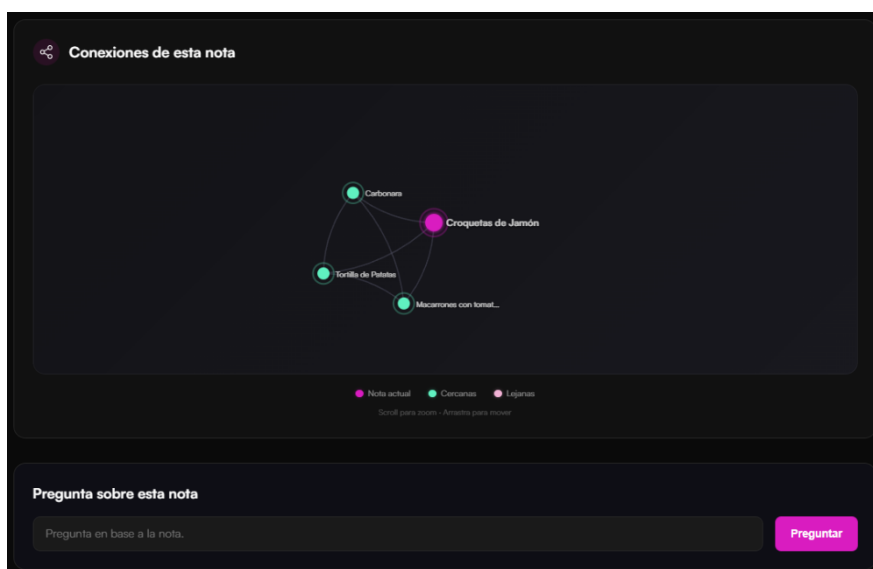


Figura 9 - Subgrafo de conexiones de una nota

- **Asistente:** permite realizar consultas globales al sistema sin necesidad de una nota origen de contexto.

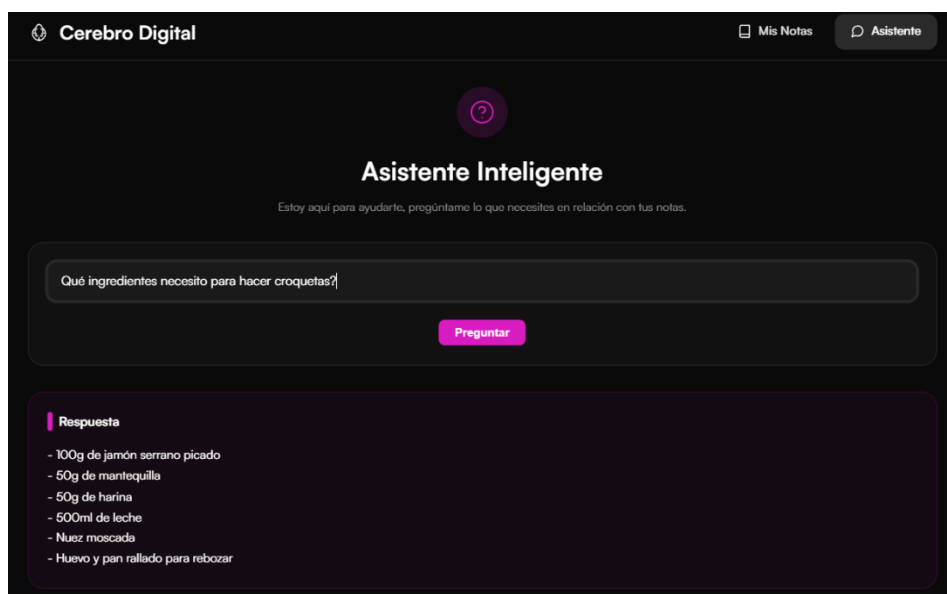


Figura 10 - Vista del asistente global

Esta organización combina dos formas de interacción complementarias: la exploración directa de las notas y la consulta asistida mediante lenguaje natural

5.2.3. Comunicación con el backend

La comunicación con el *backend* se realiza mediante peticiones HTTP asíncronas utilizando la API nativa `fetch`. Para mantener el código ordenado y centralizado, todas las llamadas se encapsulan en un módulo de servicios (`api.js`) que expone funciones de alto nivel como `getNotes()`, `getNote()`, `ask()` o `getGraph()`.

De este modo, se evita que los componentes gestionen directamente detalles relacionados con *endpoints* o peticiones HTTP, mejorando la legibilidad del código y facilitando su mantenimiento.

La comunicación entre cliente y servidor se realiza en formato JSON. Por ejemplo, durante una consulta al asistente, el *frontend* envía la pregunta del usuario y, opcionalmente, el identificador de la nota activa en modo contextual, recibiendo como respuesta el texto generado por el modelo.

5.2.4. Gestión del estado

El estado de los componentes se gestiona mediante *hooks* nativos de React:

- **useState:** utilizado para manejar el estado local (como el contenido del formulario, las notas cargadas o los indicadores de carga...).

- **useEffect:** empleado para ejecutar efectos secundarios de acciones, como la carga inicial de las notas al acceder a la lista o la actualización del subgrafo asociado a una nota.

5.2.5. Componentes destacados

La interfaz se construye a partir de componentes reutilizables que encapsulan funcionalidades específicas:

- **MarkdownEditor:** permite escribir el contenido de las notas en formato *markdown*, incluyendo un modo de edición y una pestaña de vista previa. Además, soporta la detección automática de etiquetas (*#etiqueta*).
- **MarkdownWithLinks:** renderiza el contenido de las notas en modo lectura, transformando los enlaces internos (*[[Título]]*) en navegación entre notas relacionadas. Esto permite acceder al detalle de una nota a partir de su título.
- **NoteGraph:** visualiza el subgrafo de conexiones de una nota utilizando la librería D3. La nota activa se resalta visualmente, mientras que sus conexiones directas e indirectas se representan de forma diferenciada. El usuario puede interactuar con el grafo arrastrando los nodos y navegando entre notas.
- **NoteForm y modal:** gestionan la creación de nuevas notas, incluyendo validación de campos obligatorios como el título y detección automática de etiquetas.

5.2.6. Integración con el backend

El *frontend* consume los siguientes endpoints expuestos por la API REST:

MÉTODO	ENDPOINT	DESCRIPCIÓN
GET	/api/notes	Obtención de todas las notas.
GET	/api/notes/{id}	Obtener una nota.
POST	/api/notes	Crear una nota.
PUT	/api/notes/{id}	Actualizar una nota.
DELETE	/api/notes/{id}	Eliminar una nota.
POST	/api/ask	Consultar al asistente.
GET	/api/notes/{id}/graph	Obtener el grafo.

Esta arquitectura cliente-servidor permite desacoplar completamente el *frontend* de la lógica de negocio, limitándose a presentar la información y delegar las operaciones al *backend* a través de la API.

Capítulo 6. LLM y enfoque RAG

El elemento diferenciador de **Cerebro Digital** es la integración de un modelo de lenguaje capaz de realizar consultas sobre la base de conocimiento del usuario. Para ello, se implementa un enfoque *Retrieval-Augmented Generation* (RAG), donde el contexto utilizado durante la generación no se obtiene mediante *embeddings* vectoriales, sino a partir de la estructura relacional del grafo de conocimiento construido a partir de las notas y sus conexiones.

6.1. Fundamentos y componentes del sistema LLM

Los modelos de lenguaje de gran tamaño *Large Language Models* (LLM) constituyen el núcleo generativo del sistema. El modelo no dispone de acceso directo a la información almacenada por el usuario; únicamente puede utilizar el contexto incluido en el *prompt* generado por el *pipeline* RAG. De este modo, las respuestas se construyen a partir de las notas recuperadas por la aplicación junto con el conocimiento general adquirido durante el entrenamiento del modelo.

La arquitectura del sistema LLM se compone de tres elementos principales:

1. Modelo

Se ha seleccionado el modelo *Qwen3-4B-Thinking* [2] en formato GGUF. A diferencia de los modelos de tipo *instruct*, la variante *Thinking* está optimizada para tareas de razonamiento, incorporando procesos de *Chain of Thought* (CoT), técnica orientada a mejorar el razonamiento secuencial en modelos de lenguaje [3]. Esto permite al modelo analizar relaciones entre notas y generar respuestas más estructuradas y justificadas. El modelo utiliza cuantización Q4_K_M, lo que reduce el consumo de memoria manteniendo un rendimiento adecuado.

2. Motor de inferencia

El motor de inferencia es el encargado de cargar el modelo en memoria y ejecutar las operaciones necesarias para generar texto. En este proyecto se utiliza **llama.cpp** debido a su portabilidad, bajo consumo de recursos y capacidad de ejecución local sin necesidad de infraestructura externa especializada.

3. API de conexión

La comunicación entre el *backend* y el motor de inferencia se realiza mediante una API compatible con OpenAI (/v1/chat/completions). El uso de este estándar simplifica la integración con el backend y facilita la sustitución futura del modelo o del motor de inferencia sin modificar la lógica de aplicación.

6.2. Diseño del pipeline

Los sistemas RAG tradicionales suelen basarse en *embeddings* y búsqueda por similitud vectorial para recuperar contexto relevante. Sin embargo, en este proyecto se ha optado por un enfoque alternativo apoyado en la estructura del grafo.

Esta decisión se fundamenta en las siguientes razones:

- **Eficiencia computacional:** se evita el coste de generar y actualizar *embeddings* cada vez que el usuario modifica sus notas.
- **Aprovechamiento del dominio:** las relaciones explícitas entre notas (enlaces y etiquetas) proporcionan una semántica contextual superior a una búsqueda puramente estadística.
- **Simplicidad:** al tratarse de una aplicación de alcance moderado, se prioriza una solución ligera y mantenible frente a infraestructuras vectoriales más complejas.

De este modo, el proceso de recuperación reutiliza directamente la estructura relacional ya presente en el sistema, convirtiendo el grafo en el núcleo del mecanismo de contexto.

6.3. Generación Aumentada por Recuperación (RAG)

El paradigma RAG surge para mitigar una limitación fundamental de los LLM: la imposibilidad de acceder a información privada o actualizada fuera de su entrenamiento. En este contexto, RAG actúa como un mecanismo de integración entre la base de conocimiento del usuario y la capacidad generativa del modelo.

El proceso se divide en tres fases:

- **Retrieval (recuperación):** se obtiene información relevante para la consulta del usuario dentro de su base de conocimiento.
- **Augmented (aumentación):** el contexto recuperado se incorpora al *prompt* enviado al modelo, enriqueciendo el contexto.

-
- **Generation (generación):** el modelo genera una respuesta basándose exclusivamente en la información proporcionada.

6.3.1. Algoritmo de recuperación

El RAGService implementa dos estrategias de recuperación dependiendo del punto de partida de la consulta. Esta dualidad permite adaptar la búsqueda al escenario de uso: consultas generales sin referencia a una nota concreta, o preguntas realizadas desde el interior de una nota específica, donde el contexto de navegación resulta relevante.

Modo global (búsqueda por palabras clave)

Cuando el usuario realiza una consulta sin encontrarse en el detalle de una nota concreta, la recuperación se basa exclusivamente en índices de texto de MongoDB sobre los campos `title` y `content`. Este proceso, implementado en el método `findRelevantNotes`, sigue los siguientes pasos:

- Una **búsqueda de texto** completo mediante el operador `$text` de MongoDB, que recorre los índices creados con complejidad $O(\log n)$.
- Un **reordenamiento** que otorga una puntuación adicional a cada nota por cada una de sus etiquetas que coincida con alguna palabra clave extraída de la consulta del usuario. Este mecanismo permite priorizar información explícitamente categorizada por el usuario.
- **Limitación del contexto:** se establece un límite de 8000 caracteres acumulados en el contenido de las notas seleccionadas, además de un tope de 12 notas. Este umbral, equivalente aproximadamente a 6000-6200 tokens (teniendo en cuenta que 1 token equivale a 1,3 caracteres en español), evita exceder la ventana de contexto del modelo, fijada en 4096 tokens.

Este enfoque permite recuperar información relevante sin necesidad de recurrir al grafo, siendo especialmente útil para consultas de carácter general o cuando el usuario no está navegando por una nota concreta.

Modo contextual (exploración de grafo)

Cuando la consulta se realiza desde el detalle de una nota concreta, el contexto se recupera mediante una exploración BFS (*Breadth-First Search*) sobre el grafo en memoria. La elección de BFS frente a otros responde a que prioriza las relaciones más cercanas a la nota origen, que suelen contener información más relevante para la consulta.

El algoritmo consta de los siguientes pasos:

1. Inicialización de estructuras de control (cola y conjunto de nodos visitados).
2. Exploración progresiva de las notas vecinas hasta alcanzar la profundidad máxima configurada.
3. Expansión mediante enlaces: para cada nota procesada, se obtienen todos los enlaces asociados (tanto entrantes como salientes) y se añaden a la cola los identificadores de las notas destino u origen, según corresponda. Esta fase captura las relaciones explícitas definidas por el usuario.
4. Expansión adicional por etiquetas compartidas: se incorporan aquellas notas que compartan al menos una etiqueta con la nota actual. Aunque no exista un enlace explícito entre ellas, la coincidencia de etiquetas actúa como una relación implícita que puede enriquecer el contexto de la consulta.
5. Recolección y limitación de resultados: el proceso se detiene al alcanzar el número máximo de nodos configurado (actualmente 15), asegurando que el contexto resultante no supere los límites de procesamiento del modelo.

Este mecanismo permite recuperar información contextual cercana, priorizando relaciones explícitas entre notas.

6.3.2. Construcción del prompt

Una vez recuperado el contexto mediante uno de los dos mecanismos descritos, el servicio construye dinámicamente el *prompt* estructurado que será enviado al modelo de lenguaje.

En el caso de una consulta contextual, la estructura del prompt incluye:

- **Instrucciones iniciales:** un texto breve indica al modelo que debe responder exclusivamente en español y utilizando únicamente la información de las notas proporcionadas.

-
- **Nota actual (si existe):** se destaca como elemento de máxima prioridad.
 - **Notas relacionadas:** a continuación, se listan las notas recuperadas durante la exploración, que proporcionan contexto adicional.
 - **Pregunta del usuario:** se incluye la consulta original.
 - **Instrucción final:** se reitera que la respuesta debe basarse únicamente en el contexto proporcionado.

Para evitar exceder la ventana de contexto del modelo (4096 tokens), se limita a 5 el número de notas relacionadas incluidas en el prompt. Adicionalmente, el contenido de cada nota se trunca a 200-300 caracteres, y el tamaño total del prompt se controla mediante un límite de 8000 caracteres en la fase de recuperación global.

6.4. Arquitectura de integración

La integración del *pipeline* RAG mantiene la separación de responsabilidades definida en la arquitectura general del sistema.

- RAGHandler gestiona las peticiones HTTP.
- RAGService orquesta la recuperación de contexto, construcción del prompt y llamada al modelo.
- La interfaz LLMClient desacopla la lógica de aplicación de la implementación concreta del modelo.
- La capa de infraestructura implementa la comunicación HTTP con el servidor llama.cpp: gestionando la serialización JSON, los *timeouts* de red y la extracción del razonamiento de la respuesta.

Cliente LLM

El cliente LLM, ubicado en `infrastructure/llm/client.go`, actúa como adaptador encargado de comunicarse con el servidor llama.cpp.

El flujo de inferencia consiste en:

1. Recibe un *prompt* y un contexto.

-
2. Construcción de la petición ChatRequest que incluye el mensaje del usuario, las instrucciones al modelo, y los siguientes parámetros:
 - **Temperatura:** controla la aleatoriedad de la generación, un valor entre $[0, 1]$. Se ha seleccionado 0.5 para obtener respuestas precisas y coherentes, evitando invenciones.
 - **MaxTokens:** límite de la respuesta en *tokens* (establecido en 1024), suficiente para respuestas concisas.
 3. Serialización del *payload* a JSON.
 4. Envío de la petición HTTP al *endpoint* `/v1/chat/completions`.
 5. Procesamiento de la respuesta generada por el modelo, extrayendo el contenido (respuesta final o razonamiento).

La inicialización del cliente se realiza durante el arranque de la aplicación y se inyecta posteriormente en RAGService.

6.5. Limitaciones y decisiones de diseño

El enfoque adoptado presenta ciertas limitaciones que han sido asumidas durante el diseño del sistema:

- **Ausencia de búsqueda semántica:** al no utilizar *embeddings*, el sistema no puede recuperar notas semánticamente similares que no compartan palabras clave explícitas.
- **Dependencia de la calidad de los metadatos:** la eficacia de la recuperación por etiquetas y enlaces depende de la información que proporcione el usuario y del modo en que lo organice.
- **Profundidad de exploración limitada:** la exploración BFS se restringe a dos niveles de profundidad para controlar el tamaño del contexto. En algunos casos, esto podría dejar fuera información relevante situada a mayor distancia en el grafo.
- **Carga inicial del grafo:** para sistemas con un gran volumen de notas, la reconstrucción del grafo en memoria podría resultar costosa.

Dado el alcance de este proyecto, se asume una carga de trabajo moderada, asumiendo una base de datos de hasta 300 notas, un rango en el que las limitaciones descritas no afectan significativamente al rendimiento ni a la usabilidad, por lo que las limitaciones indicadas se consideran aceptables.

Capítulo 7. Verificación y Validación del Software

Con el objetivo de garantizar la corrección, robustez y mantenibilidad a largo plazo del sistema, se ha diseñado una estrategia de pruebas multinivel. Esta estrategia se divide principalmente en **pruebas unitarias**, centradas en la lógica pura del dominio, y **pruebas de integración**, orientadas a validar la orquestación entre componentes e infraestructura.

7.1. Pruebas unitarias

La validación del sistema comienza en la capa de dominio. El objetivo es asegurar que las reglas de negocio y las invariantes del modelo se cumplan de forma consistente, permaneciendo independientes a cualquier detalle de infraestructura (como bases de datos o servicios externos). Este aislamiento permite detectar errores de lógica en las fases más tempranas del desarrollo, reduciendo bruscamente el coste de corrección.

Desde el punto de vista más metodológico, las pruebas se han diseñado combinando técnicas de caja blanca y caja negra:

- **Enfoque de caja negra:** se identifica el comportamiento observable mediante el Análisis de Valores Límite (AVL) y la Partición de equivalencia. Esto permite seleccionar casos representativos de entradas válidas e inválidas (identificadores vacíos, referencias nulas, duplicados...).
- **Enfoque de caja blanca:** se adquiere un enfoque sobre el flujo de control interno del código (validaciones, condicionales, bucles). Esto permite diseñar casos que aseguren la ejecución de caminos críticos del código.

7.1.1. Prácticas de implementación en Go

Para la implementación se ha utilizado el paquete nativo `testing` de Go [4], destacando las siguientes prácticas seguidas:

1. **Table-Driven Tests:** uso de este patrón para definir los múltiples escenarios de forma declarativa, facilitando la escalabilidad y lectura del código [5].
2. **Granularidad con subtests:** empleo de `t.Run` permite el aislamiento de escenarios. Esto garantiza una trazabilidad precisa; si falla la validación de un id

vacío, no se detiene la ejecución sino que permite observar el estado completo de la suite en una sola ejecución.

3. Gestión de errores: distinción entre el uso de `t.Errorf` y `t.Fatalf` en el reporte de fallos:

- **t.Errorf:** utilizado en las tablas de pruebas para reportar que un caso concreto ha fallado, permitiendo que el bucle continúe ejecutando el resto de escenarios (no hay dependencias entre casos de prueba).
- **t.Fatalf:** reservado para errores críticos de configuración o precondition (`setUp`). Por ejemplo, en el *helper* `newGraphWithNotes()`, si falla la creación de una nota de prueba, `Fatalf` detiene el test inmediatamente, ya que intentar ejecutar lógica sobre un objeto no inicializado provocaría un *panic* en el sistema.

4. Funciones auxiliares (Helpers): para simplificar la preparación de escenarios base sobre los que ejecutar las pruebas, se implementan métodos como `newGraphWithNotes()` que encapsula la configuración (`setup`), separando la construcción del entorno de la validación propia del test.

5. Validación de invariantes: no solo se comprueban los valores de retorno, sino el estado interno. Se verifica, por ejemplo, que el contador de notas coincida con el número de cambios realizados y que atributos como `UpdatedAt` solo muten antes cambios reales de estado.

7.1.2. Pruebas de concurrencia y sincronización

Dado que el núcleo del sistema es un grafo de conocimiento que reside en memoria de forma compartida para todos los servicios, la gestión de la concurrencia es crucial. Se ha implementado un mecanismo de sincronización basado en `sync.RWMutex`, optimizado para escenarios con alta carga de lectura.

Para validar la seguridad de los hilos (*thread-safety*), se han desarrollado pruebas de estrés concurrentes que se ejecutan junto al detector de condiciones de carrera (*Race detector*) de Go mediante el flag `-race`. Estas pruebas simulan:

- **Escrituras concurrentes:** mediante distintas goroutines que intentan modificar nodos de manera simultánea. Por ejemplo, el test `TestGraph_Concurrency()` mediante el uso de `sync.WaitGroup`, lanza distintas goroutines que intentan insertar

nodos simultáneamente. Se valida que, tras la ejecución, el estado final del grafo sea consistente y el número de notas coincida con el número de inserciones exitosas.

- **Lectura y escritura mixta:** se simulan lectores concurrentes (RLock) mientras una *goroutine* de escritura intenta modificar el recurso. Esto valida que el sistema permite el acceso múltiple a la información sin bloqueos innecesarios, pero garantizando la exclusividad total durante la escritura, evitando inconsistencias en memoria.

7.2. Pruebas de integración

Mientras las pruebas unitarias validan la lógica intrínseca del dominio, las pruebas de integración se centran en la orquestación entre los servicios de capa de aplicación y sus dependencias. El objetivo principal es asegurar que el flujo de datos entre el sistema en memoria (grafo), la lógica de negocio y los mecanismos de persistencia sea correcto y consistente.

7.2.1. Mocks

Para garantizar que las pruebas de integración sean deterministas y no dependan del estado de sistemas externos (bases de datos, red...) se ha optado por el uso de **mocks**. Dado que el lenguaje Go no cuenta con una herramienta nativa de generación dinámica de mocks similar a *Mockito* en Java, se ha integrado la librería *testify/mock*.

Esta aproximación permite:

1. **Implementar interfaces de infraestructura:** se han creado estructuras de test como *MockNoteRepository* y *MockLinkRepository* que implementan las interfaces del dominio.
2. **Programación de expectativas:** mediante el método *.On()*, se define el comportamiento esperado de la base de datos para cada escenario, permitiendo forzar errores de persistencia o valores de retorno específicos.
3. **Verificación de invocaciones:** al finalizar cada test, se utilizan *AssertExpectations* para asegurar que el servicio además de devolver el valor correcto interactuó con la infraestructura de la manera prevista.

7.2.2. Diseño de Test Fixtures

Para reducir la redundancia y repetición del código y así garantizar un entorno limpio para cada ejecución, se ha implementado el patrón **Test Fixture** a través de funciones como `newNoteSvcMocks()` y `newGraphSvcMock()`. Estas funciones encapsulan:

- La instanciación de los repositorios simulados.
- La creación de una instancia limpia del Grafo de dominio en memoria.
- La inyección de estas dependencias en el servicio objeto de la prueba.

Asimismo, se hace uso de `t.Cleanup()`, una funcionalidad de Go que garantiza que las expectativas de los mocks se verifiquen automáticamente al cierre de cada subtest, simplificando la estructura de las funciones de prueba.

7.2.3. Casos de sincronización

Este proyecto debe mantener la consistencia entre el Grafo (memoria) y los repositorios (persistencia). Para ello, se han validado los siguientes flujos críticos:

- **Gestión de notas y rollback manual:** en la creación de notas (`CreateNote`), el servicio debe coordinar la validación de dominio, la persistencia y la inserción en el grafo. Las pruebas validan que:
 - Si la persistencia en base de datos falla, el grafo no se modifica, evitando que la memoria muestre inconsistencias frente a la base de datos.
 - Se ha implementado un **mecanismo de rollback**: si la nota se guarda en la base de datos pero su inserción en el grafo falla, el servicio invoca automáticamente al método `Delete()` del repositorio para revertir la situación inicial.
- **Integridad referencial en enlaces:** el `LinkService` depende tanto de notas como de enlaces. Las pruebas de integración aseguran que no se puedan crear relaciones si el servicio no detecta previamente la existencia de los nodos origen y destino en el grafo. Además, se valida la lógica de alias automáticos, comprobando que si el usuario no proporciona uno, el servicio recupera el título de la nota destino y lo inserta en el enlace antes de persistirlo.
- **Carga del Grafo:** `GraphService` es el encargado de reconstruir el estado del sistema al inicio (`LoadGraph()`). Las pruebas de integración simulan la recuperación masiva de datos desde los repositorios. Un aspecto destacable es la validación de

errores en cascada: si la carga de notas falla, el servicio debe abortar la carga de enlaces para prevenir la inconsistencia de referencias huérfanas en el sistema.

7.2.4. Inyección de generador

Para facilitar las pruebas de la creación de entidades, se ha introducido el tipo `IDGenerator func() string`. Mientras que en producción el servicio utiliza identificadores únicos reales (strings hexadecimales), en el testing se inyecta una instancia que retorna strings constantes, lo que facilita su evaluación, haciendo que los tests sean independientes del generador de IDs del sistema.

Capítulo 8. Conclusiones y líneas futuras de trabajo

El desarrollo de Cerebro Digital ha permitido diseñar e implementar una aplicación capaz de combinar la gestión de notas personales y la consulta inteligente mediante grafos y modelos de lenguaje. A lo largo del proyecto se ha aplicado una arquitectura desacoplada basada en Clean Architecture, integrando un backend en Go, una interfaz web desarrollada con React y un *pipeline* RAG apoyado en una estructura de grafo.

Uno de los aspectos más novedoso del sistema es la utilización de las relaciones entre notas como mecanismo de recuperación contextual, evitando la necesidad de emplear *embeddings* y bases de datos vectoriales. Este enfoque ha permitido desarrollar una solución ligera, mantenible y completamente ejecutable en local.

Además, la integración de modelos LLM ejecutados mediante llama.cpp demuestra la viabilidad de construir asistentes inteligentes sin depender de servicios externos, preservando así el control sobre la información del usuario.

No obstante, debido al alcance definido del proyecto, determinadas funcionalidades han quedado fuera de la implementación actual, considerándolas posibles líneas futuras de trabajo:

- **Generación automática de enlaces:** incorporar mecanismos basados en IA capaces de detectar relaciones semánticas entre notas y la creación automática de enlaces.
- **Generación automática de resúmenes:** permitir la creación de resúmenes contextuales de notas o conjuntos de notas relacionados.
- **Búsqueda semántica mediante embeddings:** complementar el enfoque basado en grafo con técnicas de similitud vectorial para recuperar información semánticamente relacionada, aunque no comparta palabras clave explícitas.
- **Sugerencia automática de etiquetas:** incorporar sistemas capaces de proponer etiquetas de forma automática para mejorar la organización y recuperación de información.
- **Persistencia incremental del grafo:** evitar la reconstrucción completa del grafo en memoria durante el arranque mediante mecanismos de sincronización incremental.

- **Soporte multiusuario y sincronización remota:** extender la aplicación hacia entornos colaborativos mediante autenticación y sincronización entre dispositivos.

En conjunto, el proyecto demuestra cómo los modelos de lenguaje pueden integrarse en sistemas de gestión de conocimiento personal mediante arquitecturas ligeras y desacopladas, abriendo la puerta a futuras ampliaciones y mejoras funcionales.

Además, aunque los objetivos iniciales del proyecto se centraban en el diseño del *backend* y la integración del *pipeline* RAG, durante el desarrollo se decidió ampliar de manera progresiva el alcance de la aplicación incorporando funcionalidades que no formaban parte del planteamiento inicial. En particular, se implementó una interfaz web completa desarrollada en React, así como un entorno de despliegue basado en contenedores Docker que facilita la ejecución y distribución de la aplicación.

Estas ampliaciones permitieron transformar el proyecto desde un inicio conceptual centrado en el backend y el pipeline RAG hacia una aplicación funcional más completa, integrando tanto la capa de presentación como mecanismos de despliegue reproducible. De este modo, el resultado final ofrece una visión más cercana a un entorno real de utilización y distribución del sistema.

8.1. Aportación personal y aprendizaje adquirido

Desde una perspectiva personal, considero que este proyecto ha supuesto una experiencia de aprendizaje especialmente enriquecedora tanto a nivel técnico como formativo. Más allá del resultado obtenido, el desarrollo de Cerebro Digital me ha permitido consolidar conocimientos adquiridos durante la carrera y aplicarlos de forma práctica en el diseño e implementación de un sistema completo y funcional.

A lo largo del proyecto ha sido necesario investigar y profundizar en tecnologías y conceptos que inicialmente desconocía, como el lenguaje Go, los modelos de lenguaje (LLM) o las arquitecturas RAG. Este proceso ha contribuido a desarrollar una mayor autonomía a la hora de afrontar nuevos retos técnicos, reforzando mi capacidad de autoaprendizaje y permitiéndome comprender y aplicar nuevos conceptos progresivamente.

Del mismo modo, este trabajo ha servido para afianzar la importancia de un buen diseño software como pilar fundamental del desarrollo. No se ha buscado únicamente implementar una aplicación funcional, sino también reflexionar sobre decisiones de diseño adoptadas y

sobre la calidad del software construido, aplicando criterios de arquitectura, modularidad y buenas prácticas que permitan mantener una base sólida, coherente y escalable.

En definitiva, considero que este proyecto me ha permitido crecer tanto a nivel técnico como profesional, reforzando conocimientos previos, adquiriendo otros nuevos y desarrollando una forma de trabajo más autónoma, crítica y orientada a la calidad del software.

Bibliografía

- [1] Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- [2] Qwen Team. (2025). *Qwen3 Documentation*. Disponible en: <https://qwenlm.github.io/> (consulta: marzo de 2026).
- [3] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. V., & Zhou, D. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. Advances in Neural Information Processing Systems (NeurIPS).
- [4] The Go Authors. (2026). *Package testing*. Disponible en: <https://pkg.go.dev/testing> (consulta: febrero de 2026).
- [5] Marcel van Lohuizen. (2016). *Using Subtests and Sub-benchmarks*. The Go Blog. Disponible en: <https://go.dev/blog/subtests> (consulta: marzo de 2026).